

Evaluating MQTT QoS Reliability in AWS IoT Core for Device-to-Cloud Telemetry under Intermittent Connectivity

Uday Kiran Reddy Dodda
X25166484
MSc in Cloud Computing

12th June 2026

1 Research Problem Background

Reliable message delivery is essential to IoT telemetry but wireless devices frequently disconnect and reconnect to the cloud and the MQTT Quality of Service (QoS) level selected by the publisher imposes trade-offs between message delivery guarantees and latency and message duplication. Shvaika et al. (2025) present TBMQ, a distributed MQTT broker that leverages Kafka and Redis that delivers durable message delivery for more than one hundred million simultaneous clients and several million messages per second. However, their work describes broker throughput and persistence in high steady load, and the authors explicitly state the varying QoS levels, intermittent connectivity and varying client conditions are left as future work.

In practical IoT communications, how much does the QoS level change the delivery completeness, duplication, and latency once the devices disconnect and reconnect is a practical question and not a property specified by the protocol as such, as stressed by Gaspar et al. (2026) in their stress-test and Kashyap and Sharma (2025) in a comparative analysis of MQTT and CoAP for IoT communications. The unmeasured question here is whether AWS IoT Core provides more reliable device-to-cloud telemetry data with QoS 0 or QoS 1 when the connections are lost.

2 Research Question

How do MQTT QoS 0 and QoS 1 levels in AWS IoT Core affect message delivery, duplication and latency for device-to-cloud telemetry under controlled disconnections?

3 Justification

The quality of service (QoS) level is one of the first decisions a team of developers working on an IoT application has to make, and is typically based on protocol folklore, and not on observed behaviour in a managed broker under the variety of disconnections that real-life devices have. This study provides that evidence, continuing the reliability interest that Shvaika et al. (2025) took from the broker-scale throughput to the QoS and intermittent-connectivity scenario that they had not considered before, on AWS IoT Core. The experiment consists of five simulated devices that publish 1,000 messages at 5-second intervals each to AWS IoT Core at the two different QoS levels, 0 and 1, with message bursts and disconnection windows placed in between and repeated over several trials, and each IoT Rules routes each message to a Lambda function that writes message identifiers and timestamps to DynamoDB, where the sent and delivered records are matched on identifier and timestamp. CloudWatch metrics, including message delivery rate, duplicate rate, end-to-end latency, reconnection time, rule triggered success rate and lost message count, are included with the estimated amount of service usage incurred for each configuration. No personal data is used and the work is limited to simulated devices and synthetic telemetry, within the free-tier limits, so it does not present an ethical issue other than that of normal acceptable use.

4 Specific Items to Be Addressed

1. *QoS comparison*: compare the message delivery rate between MQTT QoS 0 and QoS 1 for simulated device-to-cloud telemetry.
2. *Duplication and latency*: measure duplicate-message occurrence and end-to-end delivery latency at each QoS level.
3. *Disconnection scenarios*: design controlled disconnections of fixed duration and message bursts across a fixed device fleet, repeated over several trials, and evaluate the reconnection behaviour that follows.
4. *Ingestion path*: study how IoT Rules routing to a Lambda function that logs identifiers and timestamps in DynamoDB preserves delivered messages, through rule-trigger success rate and lost-message count.
5. *Usage cost*: estimate the AWS service usage each QoS configuration incurs within free-tier limits.
6. *Recommendation*: derive a recommended QoS policy for low-cost AWS telemetry workloads.

Bibliography

- Amazon Web Services (2025) *AWS IoT Core Developer Guide: MQTT*. Available at: <https://docs.aws.amazon.com/iot/latest/developerguide/mqtt.html> (Accessed: 12 June 2026).
- Gaspar, L.M., Faria, J.N., Domingues, M., Famá, F., Martins, L. and Portugal, D. (2026) ‘The price of reliability: stress-testing MQTT in practical IoT communications’, *IEEE Internet of Things Magazine*, 1-7. doi: 10.1109/MIOT.2026.3681190.
- Kashyap, M. and Sharma, V. (2025) ‘A comparative analysis and implementation of CoAP and MQTT protocol for IoT communication’, *Life Cycle Reliability and Safety Engineering*, 14(4), 715-730. doi: 10.1007/s41872-025-00324-7.
- Shvaika, A., Shvaika, D., Landiak, D. and Artemchuk, V. (2025) ‘A distributed architecture for MQTT messaging: the case of TBMQ’, *Journal of Big Data*, 12(1), 224. doi: 10.1186/s40537-025-01271-x.